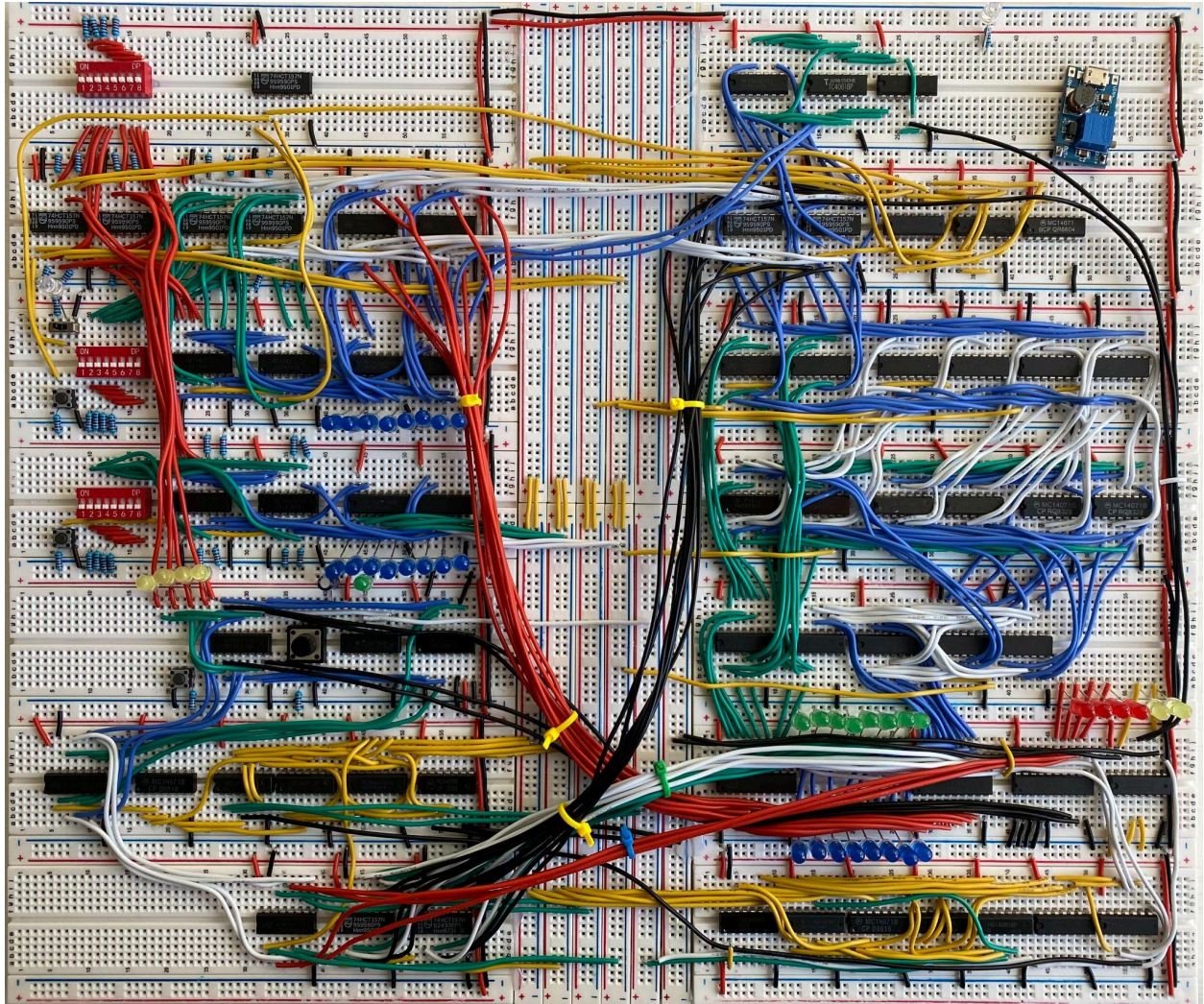


8 Bit Microprocessor Computing Problems and Programs



Nicholas Grummon, January 2022

This packet details some of the more complex computing problems that the 8-bit microprocessor is capable of solving. Almost all of these problems require a combination of simpler operations along with conditional branching to operate. They range from problems as simple as floor dividing two numbers to calculating e^x .

1. Floor Division

This operation divides two positive numbers and rounds down such that $D = X / Y$.

<u>Assembly</u>	<u>Machine</u>
X = 9	[0000] 0000 1001
Y = 3	[0001] 0000 0011
D = 0	[0010] 0000 0000
n = 1	[0011] 0000 0001
(R)	[0100]
const 0	[0101] 0000 0000
LDA #X	(0000) 0101 0000
STA #R	(0001) 0001 0100
LOOP: LDA #R	(0010) 0101 0100
LDB #Y	(0011) 0110 0001
SUB	(0100) 0000 0001
STA #R	(0101) 0001 0100
LDB #0	(0110) 0110 0101
CMP	(0111) 0000 1001
LDA #D	(1000) 0101 0010
BLT DONE	(1001) 1100 1110
LDB #n	(1010) 0110 0011
ADD	(1011) 0000 0000
STA #D	(1100) 0001 0010
B LOOP	(1101) 1110 0010
DONE: B DONE	(1110) 1110 1110

2. Fractional Division

The microprocessor does not have floating point output, so the four msb's of regA will be the integer part and the 4 lsb's will be the fractional part of the result.

<u>Assembly</u>	<u>Machine</u>
X = 3	[0000] 0000 0011
Y = 2	[0001] 0000 0010
D = 0	[0010] 0000 0000
n = 1	[0011] 0000 0001
R = X << 4	[0100] 0011 0000
const 0	[0101] 0000 0000
LOOP: LDA #R	(0000) 0101 0100
LDB #Y	(0001) 0110 0001
SUB	(0010) 0000 0001
STA #R	(0011) 0001 0100
LDB #0	(0100) 0110 0101
CMP	(0101) 0000 1001
LDA #D	(0110) 0101 0010
BLT DONE	(0111) 1100 1100
LDB #n	(1000) 0110 0011
ADD	(1001) 0000 0000
STA #D	(1010) 0001 0010
B LOOP	(1011) 1110 0000
DONE: B DONE	(1100) 1110 1100
// same procedure as floor division but shifted to the left to get more sig figs	

3. EOR Implementation

$$A \oplus B = A + B - AB$$

<u>Assembly</u>	<u>Machine</u>
X = 0xA7	[0000] 1010 0111
Y = 0xFF	[0001] 1111 1111
(P) (Q)	[0010] [0011]
LDA #X	(0000) 0101 0000
LDB #Y	(0001) 0110 0001
OR	(0010) 0000 0011
STA #P	(0011) 0001 0010
LDA #X	(0100) 0101 0000
AND	(0101) 0000 0010
STA #Q	(0110) 0001 0011
LDA #P	(0111) 0101 0010
LDB #Q	(1000) 0110 0011
SUB	(1001) 0000 0001

5. Left Shift Implementation

<u>Assembly</u>	<u>Machine</u>
A = 0b11010110	[0000] 1101 0110
N = 3	[0001] 0000 0011
X = 0	[0010] 0000 0000
I = 1	[0011] 0000 0001
LOOP: LDA #X	(0000) 0101 0010
LDB #N	(0001) 0110 0001
CMP	(0010) 0000 1001
BLT CONT	(0011) 1100 0101
B OUT	(0100) 1110 1101
CONT: LDB #I	(0101) 0110 0011
ADD	(0110) 0000 0000
STA #X	(0111) 0001 0010
LDA #A	(1000) 0101 0000
LDB #A	(1001) 0110 0000
ADD	(1010) 0000 0000
STA #A	(1011) 0001 0100
B LOOP	(1100) 1110 0000
OUT: LDA #A	(1101) 0101 0100
DONE: B DONE	(1110) 1110 1101

4. Fibonacci Sequence

$$A_n = A_{n-1} + A_{n-2}$$

<u>Assembly</u>	<u>Machine</u>
X = 1	[0000] 0000 0001
Y = 0	[0001] 0000 0000
(Z)	[0010]
LDA #X	(0000) 0101 0000
LOOP: LDB #Y	(0001) 0110 0001
STA #Y	(0010) 0001 0001
ADD	(0011) 0000 0000
B LOOP	(0100) 1110 0001

6. Rotate Implementation

<u>Assembly</u>	<u>Machine</u>
A = 0b11010110	
N = 3	
X = 0	
I = 1	
Z = 0	
LOOP: LDA #X	
LDB #N	
CMP	
BLT CONT	
LDA #A	
DONE: B DONE	
CONT: LDB #I	
ADD	
STA #X	
LDA #A	
LDB #Z	
CMP	
LDB #A	
ADD	
BLT ROLL	
B NORM	
ROLL: LDB #I	
ADD	
NORM: STA #A	
B LOOP	